

Experiment No: 01

Name of experiment: Abstract classes vs Interface.

Objective:

Understand the differences between abstract classes and interfaces, particularly in terms of multiple inheritance and know when to use each.

Conceptual Comparison:

Feature	Abstract class	Interface
Multiple inheritance	A class can inherit one abstract class	A class can implement multiple interfaces.
Methods	can have abstract and concrete methods	All methods are abstract
Fields	can have instance variables.	only public static final constants
constructor	can have constructor.	cannot have constructor.

When to use : ~~02~~

Abstract class:

- (i) When you want to share common code among related classes.
- (ii) When you need instance variable or constructors.

Interface:

- (i) When you want unrelated classes to follow the same contract.
- (ii) When you want multiple inheritance of type.

Code example:

// Abstract class example

```
abstract class variable vehicle {  
    String brand;  
    vehicle(String brand) {  
        this.brand = brand; }  
    abstract void start();  
    void showBrand() {  
        System.out.println("Brand:" + brand);  
    }  
}
```

Interface Electric {

```
    void charge();  
}
```


~~Inheritance~~

```
interface GPS {  
    void navigate();  
}
```

class Tesla extends vehicle implements ElectricGPS {

```
    Tesla(String brand) {  
        super(brand);  
    }
```

```
    void start() {  
        System.out.println("Tesla started");  
    }
```

```
    public void charge() {  
        System.out.println("charge Tesla");  
    }
```

```
    public void navigate() {  
        System.out.println("Navigating using GPS");  
    }  
}
```

```
public class Lab4 {
```

```
    public static void main(String[] args) {
```

```
        Tesla mycar = new Tesla("Tesla Model S");
```

```
        mycar.showBrand();
```

```
        mycar.start();
```

```
        mycar.charge();
```

```
        mycar.navigate();  
    }  
}
```


Output:

Brand: Tesla Model S
Tesla started silently.
Charging Tesla battery.
Navigating using GPS.

Conclusion:

- (i) Abstract classes are best for related classes sharing common code.
- (ii) Interfaces are ideal for multiple inheritance and defining contracts for unrelated classes.
- (iii) In practice, a class can extend one abstract class and implement multiple interfaces to get benefit of both.

Experiment NO: 02

Name of experiment: Encapsulation for Data Security and Integrity.

Objectives:

Understand how encapsulation protects data security and ensures data integrity by restricting direct access to class variables and providing controlled access through validated methods.

Concept of Encapsulation:

(i) Encapsulation is the process of hiding the internal state of an object and providing public methods to access and modify it.

(ii) It ensures:

a) Data security b) Data integrity

Key principles:

(i) class variables are declared private.

(ii) Public getter and setter methods provide controlled access.

(iii) Validation logic in setters prevents invalid data entry.

Example: BankAccount class:

```
public class BankAccount {  
    private String accountNumber;  
    private double balance;  
    //setters  
    public void setAccountNumber(String accountNumber)  
    {  
        if (accountNumber == Null || accountNumber  
            isEmpty)  
        {  
            System.out.println("Invalid");  
        }  
        else if  
        {  
            this.accountNumber = accountNumber;  
            System.out.println("Acc number set  
                successfully");  
        }  
    }  
    public void setInitialBalance(double balance)  
    {  
        if (balance < 0)  
        {  
            System.out.println("Invalid balance");  
        }  
        else if  
        {  
            this.balance = balance;  
            System.out.println("Initial balance  
                set successfully");  
        }  
    }  
}
```



```
public String getAccountNumber() {  
    return accountNumber; }  
public double getBalance() {  
    return balance; }  
// Deposit method
```

```
public void deposit(double amount) {  
    if (amount <= 0) {  
        System.out.println("Deposit amount must  
            be positive!"); }  
    else {  
        balance += amount;  
        System.out.println("Deposited: " + amount  
            + " New balance: " + balance);  
    }  
}
```

```
public void withdraw(double amount) {  
    if (amount <= 0) {  
        System.out.println("Withdrawal  
            amount must be positive!"); }  
    else if (amount > balance) {  
        System.out.println("Insufficient  
            balance."); }  
    else {  
        balance -= amount;  
        System.out.println("Withdrawn: " +  
            amount + " Remaining balance: " +  
            balance);  
    }  
}
```



```

public class Lab24
{
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.setAccountNumber("12345");
        account.setInitialBalance(1000);
        account.deposit(500);
        account.withdraw(200);
        account.deposit(-50);
        account.setAccountNumber("");
        account.setInitialBalance(-100);
        System.out.println("Final Account Number: " + account.getAccountNumber());
        System.out.println("Final Balance: " + account.getBalance());
    }
}

```

Output: (Example):

Account number set successfully.
Initial balance set successfully.

Deposited : 500.0, New balance: 1500.0

Withdrawn : 200.0, Remaining balance: 1300.0

Deposit amount must be positive.

Invalid account number.

Invalid balance.

Flow Encapsulation Ensures Security and Integrity:

- (1) Private variables: Prevent direct modification of sensitive data.
- (2) Validated methods: Reject null, empty or negative values to maintain data integrity.

Conclusion: Encapsulation ensures the Bank Account data cannot be corrupted and is safe from unauthorized access.

Experiment NO: 03

Name of experiment: Multithreading
Based Car Parking Management system.

Objective:

To simulate a car parking system using
multithreading and synchronization,

Requirements:

Java, Threads, synchronization,

Working procedure:

- (1) create a shared parking pool.
- (2) Generate multiple car requests.
- (3) Start multiple parking agent threads.
- (4) Agents park cars one by one.

Codes: (RegistrarParking.java)

```
class RegistrarParking{
```

```
String carNumber;
```

```
RegistrarParking(String carNumber){
```

```
    this.carNumber = carNumber;
```

```
    System.out.println("car" + carNumber +  
                        "requested parking");
```

```
}
```

ParkingPool.java:

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
class ParkingPool {
```

```
    Queue<RegistrarParking> queue = new  
        LinkedList<>();
```

```
    synchronized void addCar (RegistrarParking  
        car) {
```

```
        queue.add(car);
```

```
        notify(); }
```

```
    synchronized RegistrarParking getCar()
```

```
        throws InterruptedException {
```

```
        while (queue.isEmpty()) {
```

```
            wait(); }
```

```
        return queue.poll(); }
```

```
}
```

ParkingAgent.java:

```
class ParkingAgent extends Thread {
```

```
    ParkingPool pool; int agentId;
```

```
    ParkingAgent (ParkingPool, int agentId) {
```

```
        this.pool = pool;
```

```
        this.agentId = agentId;
```

```
    public void run() {
```

```
        try {
```



```

while(true){
    RegistrarParking car = pool.getCar();
    System.out.println("Agent"+agentId+
        "parked car"+car.CarNumber);
    Thread.sleep(500);
} catch (InterruptedException e){
    e.printStackTrace();
}
}

```

MainClass.java:

```

public class MainClass{
    public static void main(String[] args){
        ParkingPool pool = new ParkingPool();
        ParkingAgent a1 = new ParkingAgent(pool, 1);
        ParkingAgent a2 = new ParkingAgent(pool, 2);
        a1.start();
        a2.start();
        String[] cars = {"ABC123", "XYZ456", "LMN789"};
        for (String car : cars) {
            pool.addCar(new RegistrarParking(car));
        }
    }
}

```

Output: Car ABC123 requested parking.

Car XYZ456 requested parking.

Agent 1 parked car ABC123.

Agent 2 parked car XYZ456.

Conclusion:

This lab demonstrates multithreading and synchronization using the producer-consumer model in java.

Experiment No: 04

Name of Experiment: JDBC communication with Relational Database:

Objective:

To study how JDBC enables communication between a Java application and a relational database and execute a select query.

JDBC Communication management:

- (1) Java application uses JDBC API.
- (2) JDBC driver converts Java calls to database-specific calls.
- (3) Database executes SQL and returns result
- (4) JDBC sends results back to Java application

Steps to Execute a Select Query:

- (1) Load JDBC driver.
- (2) Establish database connection
- (3) Create SQL statement
- (4) Execute SELECT query.
- (5) Fetch data using ResultSet
- (6) Close resources.

Codes:

```
import java.sql.*;  
public class JdbcSelectDemo {  
    public static void main(String[] args) {  
        Connection con = null;
```



```
Statement stmt = null;
```

```
Result rs = null;
```

```
try { // Load driver
```

```
    class.forName("com.mysql.jdbc.Driver");
```

```
    // Create connection
```

```
    con = DriverManager.getConnection {
```

```
        "jdbc:mysql://localhost:3306/studentdb",
```

```
        "root", "password");
```

```
    stmt = con.createStatement();
```

```
    rs = stmt.executeQuery("SELECT * FROM  
                             student");
```

```
    // Fetch results
```

```
    while(rs.next()) {
```

```
        System.out.println(rs.getInt("id") +
```

```
        " " + rs.getString("name"));
```

```
    }
```

```
    { catch (Exception e) {
```

```
        System.out.println("Error: " + e);
```

```
    } finally {
```

```
        try { if(rs != null) rs.close();
```

```
            if(stmt != null) stmt.close();
```

```
            if(con != null) con.close();
```

```
        } catch (SQLException e) {
```

```
            System.out.println("closing error:  
                                " + e);
```

```
    }
```


Error Handling:

(I) try : database operations.

(II) catch : handles SQL and runtime errors.

(III) finally : closes connection and resources.

Conclusion:

JDBC provides a standard way for Java application to interact with relational databases using SQL queries and proper exception handling.

Experiment No: 05

Name of experiment: Servlet Controller in Java EE (MVC Pattern).

Objective:

To understand how a servlet controller manages communication between the model and the view.

Role of Servlet Controller:

- (1) Acts as the Controller in MVC.
- (2) Receives client request.
- (3) Interacts with the Model.
- (4) Sends data to the view (JSP).

Flow of Execution:

- (1) Client sends request.
- (2) Servlet processes request.
- (3) Servlet gets data from model.
- (4) Data stored as request attribute.
- (5) Request forwarded to JSP
- (6) JSP renders response.

Code:

```
import java.io.*;  
import javax.servlet.*;  
import javax.servlet.http.*;
```



```
public class studentservlet extends  
    HttpServlet  
protected void doGet(HttpServletRequest  
    request, HttpServletResponse response)  
    throws ServletException, IOException
```

```
// Model data
```

```
String name = "Rohan";
```

```
int marks = 85;
```

```
request.setAttribute("studentName",  
    name);
```

```
request.setAttribute("studentMark",  
    marks);
```

```
// Forward to JSP
```

```
request.getRequestDispatcher("result.jsp").forward(request, response);
```

```
{  
}
```

```
JSP:
```

```
<html>
```

```
<body>
```

```
<h2>Student Result </h2>
```



```
Name: ${studentName}<br>
Marks: ${studentMarks}
</body>
</html>
```

conclusion: The servlet controller manages application flow by linking the model and view using request forwarding.

```
Name: ${studentName}<br>
Marks: ${studentMarks}
</body>
</html>
```

conclusion: The servlet controller manages application flow by linking the model and view using request forwarding.

Experiment No: 06

Name of experiment: Prepared Statement in JDBC.

Objective:

To understand how Prepared Statement improves performance and security over Statement in JDBC.

Prepared Statement vs Statement

Performance:

- ① SQL query is precompiled.
- ② Can be reused multiple times.
- ③ Faster for repeated execution.

Security:

- ① Prevents SQL Injection.
- ② Parameters are safely handled.
- ③ No direct string concatenation.

Code:

```
import java.sql.*;  
  
public class InsertStudent {  
    public static void main(String[] args) {
```

```
Connection con=null;
```

```
PreparedStatement ps=null;
```

```
try {
```

```
    Class.forName("com.mysql.cj.jdbc.  
                    Driver");
```

```
    con=DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/student",  
        "root", "password");
```

```
    String sql="INSERT INTO student(id,  
                    name, marks) VALUES  
                    (?,?,?)";
```

```
    ps=con.prepareStatement(sql);
```

```
    ps.setInt(1, 101);
```

```
    ps.setString(2, "Rohan");
```

```
    ps.setInt(3, 90);
```

```
    ps.executeUpdate();
```

```
    System.out.println("Record inserted  
                        successfully");
```

```
    } catch (Exception e) {
```

```
        System.out.println("Error: " + e);
```

```
    }
```



```
finally { try {
```

```
    if (ps != null) ps.close();
```

```
    if (con != null) con.close();
```

```
} catch (SQLException e) {
```

```
    System.out.println(e);
```

Result:

The record was inserted securely and efficiently using PreparedStatement.

Conclusion:

PreparedStatement improves JDBC applications by increasing performance, enhancing security and simplifying parameter handling.

Experiment No: 07

Name of experiment: ResultSet in JDBC.

Introduction:

ResultSet stores data returned by a SELECT query. It acts like a cursor pointing to database rows.

Objective:

To understand ResultSet and its use in retrieving data from a MySQL database.

ResultSet methods:

- ① ~~Next~~ next()
- ② getString()
- ③ getInt()

Code:

```
import java.sql.*;  
public class ResultSetDemo {  
    public static void main(String[] args) {  
        try {  
            Class.forName("com.mysql.cj.jdbc.  
                Driver");  
        }  
    }  
}
```



```
Connection con = DriverManager.getConnection  
( "jdbc:mysql://localhost:3306/studentdb",  
  "root", "password");
```

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT id, name, marks from  
student");
```

```
while(rs.next())  
{  
    int id = rs.getInt("id");  
    String name = rs.getString("name");  
    int marks = rs.getInt("marks");  
    System.out.println(id + " " + name + " "  
                        + marks);  
}
```

```
con.close();
```

```
catch (Exception e){  
    System.out.println(e);  
}
```

```
}
```

```
}
```

Result: ↓

Records were successfully retrieved from the database.

Conclusion:

ResultSet allows Java programs to access database records efficiently using cursor based methods.

Experiment No: 08

Name of experiment: CRUD Application using Spring Boot and MySQL.

Objective:

To design a simple CRUD application to manage student records using Spring Boot and MySQL.

Entity(student):

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class student {
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType  
IDENTITY)
```

```
private int id;
```

```
private String name;
```

```
private int marks;
```

```
// getters and setters
```

```
}
```

Repository Interface:

```
import org.springframework.data.jpa.repository.  
JpaRepository
```

```
public interface StudentRepository extends  
    JpaRepository<Student, Integer>
```

/*

CRUD operations:

create:

```
studentRepo.save(student);
```

Read:

```
studentRepo.findAll();
```

```
studentRepo.findById(id);
```

Update:

```
student.setMarks(90);
```

```
studentRepo.save(student);
```

Delete:

```
studentRepo.deleteById(id);
```

Controller:

```
public class StudentController {
```

```
@Autowired
```

```
    StudentRepository studentRepo;
```

```
@PostMapping
```

```
    public Student addStudent(@RequestBody  
                                Student s) {
```



```
return studentRepo.save(s);
```

```
@GetMapping
```

```
public List<Student> getAll()
```

```
{  
    return studentRepo.findAll();  
}
```

```
@PutMapping("/{id}")
```

```
public Student update(@PathVariable
```

```
int id, @RequestBody
```

```
Student s)
```

```
{  
    s.setId(id);
```

```
    return studentRepo.save(s);
```

```
}
```

```
public void delete(@PathVariable int id)
```

```
{  
    studentRepo.deleteById(id);
```

```
}
```

Conclusion:

Spring Boot with Spring Data JPA

simplifies CRUD operations using

repository interfaces and annotations.

Experiment No. 089

Name of experiment: RESTful services
Using Spring Boot.

Objective:

To study how Spring Boot simplifies the development of RESTful web services.

How Spring Boot simplifies REST development:

- ① Auto-configuration.
- ② Embedded server.
- ③ Easy dependency management
- ④ Built-in JSON support.

Annotations Used:

- ① @RestController.
- ② @GetMapping.
- ③ @PostMapping.
- ④ @RequestBody.

Codes:

Student.java:

```
public class Student {  
    private int id;  
    private String name;  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getName() { return name; }  
    public void setName(String name) {  
        this.name = name; }  
}
```

StudentController.java:

```
import org.springframework.web.bind.  
    annotation.*;
```

@RestController

@RequestMapping("/student")

```
public class StudentController {
```

@GetMapping

```
    public Student getStudent() {  
        Student s = new Student();  
        s.setId(1);  
    }
```

```
s.setName("Rohan");  
return s;
```

}

② PostMapping

```
public String addStudent(@RequestBody  
Student s) {
```

```
return "student added: " + s.getName();
```

```
}
```

JSON Handling:

- ① GET return Java object ~~as~~ as JSON
- ② POST accepts JSON and maps it to Java object.

Sample JSON:

```
{  
  "id": 2,  
  "name": "Ayaan"  
}
```

Result: REST APIs we created successfully
Conclusion: Spring Boot simplifies RESTful
service development using annotations,
auto configurations and built in JSON support.